// T-OLIMPIANS — TUTORIAL

Trwałe drzewo przedziałowe z lazy propagation

Persistent segment tree na wskaźnikach w C++. Path copying, lazy bez push-downu, inwariant, gotowy kod i test poprawności.

C++ · WSKAŹNIKI · STRUKTURY DANYCH · OI / OIJ

T-Olimpians Team Poland · t-olimpians.com

01 Po co nam trwałe drzewo?

> Zwykłe drzewo przedziałowe po aktualizacji **zapomina** poprzedni stan. Wersja trwała (*persistent*) trzyma **wszystkie** stany naraz i pozwala pytać o dowolny z nich.

HISTORIA WERSJI

```
v0 = build({1,2,3,4,5,6,7,8});
v1 = rangeAdd(v0, 1, 4, +10);    // v0 dalej istnieje i dziala
v2 = rangeAdd(v1, 0, 7, -1);
v3 = rangeAdd(v1, 5, 6, +100);   // galaz z v1, NIE z v2
query(v0, 0, 7);                 // 36 - tak, jakby nic sie nie stalo
```

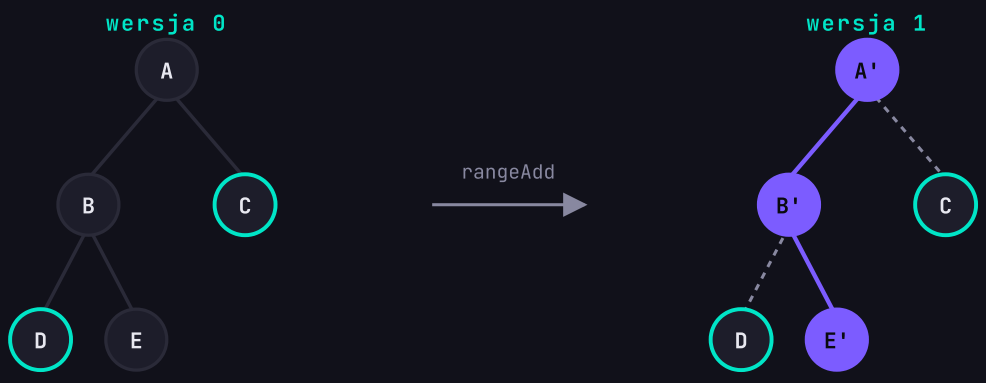
Zwróć uwagę na **v3**: nową wersję można zrobić z *dowolnej* starej, nie tylko z ostatniej. Historia wersji jest więc **drzewem**, a nie listą — stąd bierze się siła tej struktury (cofanie, „co by było gdyby", zapytania offline po czasie).

Operacja	Znaczenie	Koszt
`build(a)`	wersja 0 z tablicy	`O(n)` czasu i pamięci
`rangeAdd(ver, l, r, v)`	nowa wersja: `a[i] += v` dla $i \in [l, r]$	`O(log n)` czasu i **nowej** pamięci
`query(ver, l, r)`	suma `a[l..r]` w wersji `ver`	`O(log n)` czasu, **0** nowej pamięci

> **Naiwne podejście:** kopiowanie całej tablicy przy każdej wersji to `O(n)` pamięci na wersję. Dla `n = q = 100 000` to 10^{10} komórek — kilkadziesiąt gigabajtów. Trwałe drzewo zbija to do `O(n + q log n)`.

02 Trwałość = path copying

Podstawowy trik: żeby coś zmienić, **nie modyfikujemy żadnego starego węzła**. Kopiujemy tylko węzły na ścieżce od korzenia w dół, a nietknięte poddrzewa nowa wersja **współdzieli** ze starą — po prostu ten sam wskaźnik.



nowe węzły — powstają tylko na ścieżce, O(log n) sztuk na aktualizację

węzły współdzielone — dokładnie ten sam wskaźnik w obu wersjach

Ścieżka od korzenia do liścia ma długość **log n** — i dokładnie tyle nowych węzłów powstaje. Cała reszta drzewa jest wspólna dla obu wersji, bo **nikt jej nie nadpisuje**. W kodzie widać to w jednej linijce:

SERCE TRWAŁOŚCI

```
// brak przeciecia z [l, r] -> oddajemy STARY wskaznik.
// zero nowych wezlow, cale poddrzewo wspoldzielone przez obie wersje
if (r < lo || hi < l) return v;
```

Zapamiętaj: każda operacja pisząca zaczyna się od `clone` węzła. Jeżeli gdziekolwiek zapiszesz do węzła, którego nie sklonowałeś w tej operacji — właśnie zepsułeś wszystkie starsze wersje.

03 Gdzie lazy zderza się z trwałością

W zwykłym drzewie lazy działa tak: schodząc przez węzeł robimy **push-down** — spychamy zaległą operację do dzieci. To jest **zapis do dzieci**.

I tu robi się problem. W drzewie trwałym dzieci są **współdzielone ze starymi wersjami**. Push-down zapisałby lazy do węzła, który należy też do wersji 0 — i wersja 0 nagle zmieniłaby wartości. Cała trwałość w gruzach.

Pułapka: push-down robimy także w **zapytaniach**. W trwałym drzewie oznaczałoby to, że samo *czytanie* starej wersji ją psuje — a błąd zobaczysz dopiero w zapytaniu zadanym 200 operacji później.

W trwałym drzewie przedziałowym z lazy **nie ma push-downu.**

Skoro nie możemy zepchnąć lazy w dół, to niech tam zostanie — a ten, kto schodzi niżej, sam **pamięta**, co go po drodze „dotyczy". Cała reszta tego arkusza to konsekwencje tej jednej decyzji.

04 Inwariant — to jest cały algorytm

Pole	Co dokładnie oznacza
`v->sum`	prawdziwa suma przedziału węzła `v`, **z uwzględnieniem własnego** `v->lazy`, ale **bez** lazy przodków `v`
`v->lazy`	„+lazy do każdego elementu", które **nie jest widoczne w dzieciach** i nigdy do nich nie trafi

```
v->sum = v->l->sum + v->r->sum + v->lazy × ilość wierzchołków w poddrzewie v
```

> **ilość wierzchołków w poddrzewie v** — czyli ile pozycji tablicy obsługuje węzeł `v`; w kodzie jest to po prostu `hi - lo + 1`.

Ostatni składnik trzeba doliczać **ręcznie** — dzieci o własnym lazy rodzica nie wiedzą i nigdy się nie dowiedzą. Kto o tym zapomni, dostanie drzewo, które zgadza się w liściach i rozjeżdża w środku.

> **Konsekwencja:** skoro lazy nigdy nie schodzi w dół, to węzeł raz zapisany jest **niezmienny** (immutable). Dlatego można go bezpiecznie współdzielić między dowolną liczbą wersji.

05 Węzeł i aktualizacja

STRUKTURA WĘZŁA

```
struct Node {
    ll sum;         // suma przedzialu (z wlasnym lazy, bez lazy przodkow)
    ll lazy;        // zalegle +lazy dla kazdego elementu, niewidoczne w dzieciach
    Node *l, *r;    // dzieci - WSPOLDZIELONE miedzy wersjami!

    Node(ll s, ll lz, Node* L, Node* R) : sum(s), lazy(lz), l(L), r(R) {}
};

// kopia PLYTKA: dzieci zostaja te same wskazniki co w starej wersji
Node* cloneNode(const Node* v) {
    return new Node(v->sum, v->lazy, v->l, v->r);
}
```

NOWA WERSJA W O(LOG N)

```
Node* rangeAdd(Node* v, int lo, int hi, int l, int r, ll val) {
    if (r < lo || hi < l) return v;          // wspoldzielenie calego poddrzewa

    Node* nv = cloneNode(v);                 // nigdy nie piszemy po starym wezle!

    if (l <= lo && hi <= r) {                // PELNE pokrycie - konczymy tutaj
        nv->sum  += val * (hi - lo + 1);     // sum i lazy ruszamy RAZEM,
        nv->lazy += val;                     // inaczej inwariant pada
        return nv;
    }

    int mid = lo + (hi - lo) / 2;
    nv->l = rangeAdd(v->l, lo,      mid, l, r, val);
    nv->r = rangeAdd(v->r, mid + 1, hi,  l, r, val);
    nv->sum = nv->l->sum + nv->r->sum + nv->lazy * (ll)(hi - lo + 1);
    return nv;
}
```

> Przy pełnym pokryciu **nie schodzimy niżej** — dzieci zostają dokładnie te ze starej wersji, a informacja „temu poddrzewu należy się +val" siedzi w `nv->lazy`. Ostatnia linijka to inwariant z sekcji 04 przepisany 1:1.

06 Zapytanie — lazy zbierane do akumulatora

Skoro lazy zostaje w węźle, to schodząc w dół zbieramy lazy **wszystkich przodków** do akumulatora `acc` i doliczamy je dopiero na dole.

ZAPYTANIE, KTÓRE NICZEGO NIE PSUJE

```
ll query(Node* v, int lo, int hi, int l, int r, ll acc) {
    if (r < lo || hi < l) return 0;
    if (l <= lo && hi <= r)
        return v->sum + acc * (ll)(hi - lo + 1);   // <- tu wpada acc

    int mid = lo + (hi - lo) / 2;
    acc += v->lazy;                                // zbieramy po drodze
    return query(v->l, lo,      mid, l, r, acc)
         + query(v->r, mid + 1, hi,  l, r, acc);
}
```

Dlaczego to poprawne. Jeśli przodek ma `lazy = +5`, to każdy element jego poddrzewa jest w rzeczywistości o 5 większy, niż wynika z zapisanych niżej sum. Gdy zatrzymamy się na fragmencie obsługującym `d` pozycji tablicy, brakuje nam dokładnie `5 · d` — i tyle dodajemy. Niczego nie trzeba zapisywać.

Miła własność: zapytanie **nie modyfikuje ani jednego węzła**. Możesz pytać o dowolną wersję, w dowolnej kolejności, dowolnie wiele razy — bez efektów ubocznych i bez alokacji.

07 Złożoność

Operacja	Czas	Nowa pamięć
`build`	`O(n)`	2n − 1 węzłów
`rangeAdd`	`O(log n)`	`O(log n)` węzłów
`query`	`O(log n)`	0

przy q aktualizacjach: `O(n + q log n)` pamięci

Dla porównania — pomiar z dołączonego programu testowego, gdzie każda kolejna operacja losuje wersję bazową z *całej* dotychczasowej historii:

Wariant	Węzły
40 elementów, 149 wersji — trwałe drzewo (path copying)	`1 739`
to samo, gdyby kopiować całe drzewo na każdą wersję	`≈ 11 800`

> Różnica rośnie z `n`: kopiowanie całości to `O(n)` na wersję, path copying — `O(log n)`.

08 Pułapki konkursowe

01 **Zapis do starego węzła.** Każde `v->coś = ...` musi iść na świeżą kopię. Jedno przeoczenie psuje wersje wstecz, a debugowanie tego jest okropne.

02 **Push-down.** Ani w aktualizacji, ani — zwłaszcza — w zapytaniu.

03 `sum` **bez** `lazy`**.** Przy pełnym pokryciu ruszasz oba pola naraz: `sum += val * (hi-lo+1)` *oraz* `lazy += val`.

04 **Brak własnego lazy przy przeliczaniu.** `nv->sum = l->sum + r->sum + nv->lazy * (hi-lo+1)` — ostatni składnik znika najczęściej.

05 **Złe mnożenie** `acc`**.** Mnożysz przez ilość wierzchołków w poddrzewie, na którym się zatrzymałeś — nie przez długość całego zapytania.

06 **Pamięć.** Nie wolno robić `delete` na pojedynczym węźle — jest współdzielony. Na zawodach po prostu nie zwalniasz; poza nimi użyj areny (`std::deque<Node>` — **nie** `vector`, bo realokacja unieważni wszystkie wskaźniki) albo `shared_ptr`.

07 **Stała.** Wersja na wskaźnikach jest wolniejsza od tablicowej (`int lc[MAX], rc[MAX]`). Przy `n, q ~ 10^6` tablice bywają 2–3× szybsze i lepiej leżą w cache. Wskaźniki są za to czytelniejsze do nauki.

09 Kiedy ta technika działa

Warunek jest jeden: operacja lazy musi dać się zastosować do całego przedziału **bez zaglądania do dzieci** — znając tylko ilość wierzchołków w poddrzewie — i musi być **składalna** w akumulatorze.

	Operacja	Uwagi
✓	dodawanie na przedziale + suma	wzorcowy przypadek, opisany wyżej
✓	dodawanie + min / max	`acc` dodajesz, nie mnożysz przez ilość wierzchołków
✓	przypisanie (assign) na przedziale	`acc` to „ostatnie przypisanie od przodka”, przykrywa wszystko niżej
~	assign + add jednocześnie	działa, ale `acc` musi być parą `(assign?, add)` ze świadomą kolejnością składania
✗	operacje nieskracalne do jednej wartości	jeśli nie streścisz efektu na przedziale — lazy nie pomoże

Alternatywa: można też robić push-down, ale **tworząc przy nim nowe węzły** (klonujesz dzieci przed zapisem). Wtedy aktualizacja wygląda klasycznie, ale zapytanie musiałoby zwracać nowy korzeń albo alokować przy każdym czytaniu — dlatego wersja z akumulatorem jest standardem.

10 Gotowy kod i wynik uruchomienia

Pełna implementacja (budowa, aktualizacja, zapytanie, opakowanie na wersje oraz test losowy porównujący **każdą** wersję z brute force) leży w pliku `persistent-segtree-lazy.cpp`.

KOMPILACJA

```
g++ -std=c++14 -O2 -o pst persistent-segtree-lazy.cpp && ./pst
```

WYNIK

```
=== DEMO ===
wezlow po budowie: 15
wersja 0: [1, 2, 3, 4, 5, 6, 7, 8]
wersja 1: [1, 12, 13, 14, 15, 6, 7, 8]
wersja 2: [0, 11, 12, 13, 14, 5, 6, 7]
wersja 3: [1, 12, 13, 14, 15, 106, 107, 8]
wersja 0 (nadal nietknieta): [1, 2, 3, 4, 5, 6, 7, 8]

suma [0..7] w wersji 0 = 36  (oczek. 36)
suma [0..7] w wersji 1 = 76  (oczek. 76)
suma [0..7] w wersji 2 = 68  (oczek. 68)
suma [2..5] w wersji 3 = 148  (oczek. 148)
wezlow po 3 aktualizacjach: 30

=== TEST LOSOWY (kazda wersja porownana z brute force) ===
OK - wszystkie wersje sie zgadzaja
wersji: 149, wezlow lacznie: 1739 (drzewo jednowersyjne mialoby 79)
```

Test losowy: 300 operacji, wersja bazowa losowana z **całej** historii (więc gałęzi jest dużo), a na koniec porównanie każdej wersji element po elemencie z tablicą referencyjną. To minimum, jakie warto odpalić, zanim wyślesz trwałe drzewo na sprawdzaczkę.